

Relational Model — Chapter 2

2025 年 9
月

郭迎
计算机学院



北京邮电大学

Outline

- **Structure of Relational Databases** (**syntax**)
- **Integrity constraints** (**semantic**)
 - constraints on **schema attributes**, e.g. value domain, key
 - constraints on **dependencies** among attributes, e.g. **age** vs **birth-date**
 - constraints on **dependencies** among attributes of **different schemas**
- **Operations on relation model**, i.e. **relational algebra**

2.1 Structure of Relational Databases

- **Def: A relational database** consists of a collection of tables

- Example : a relation **instructor**

The diagram shows a table with four columns: *ID*, *name*, *dept_name*, and *salary*. The first three columns are grouped by an arrow pointing to the label "attributes (or columns)". The entire table is grouped by an arrow pointing to the label "tuples (or rows)".

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Definition of Relational Model

Modeling: elements in model

- $A_1, A_2, \dots, A_n$ are **attributes**
- $D_1, D_2, \dots, D_n$ are **domains** of attributes
 - ▣ the set of allowed values for each attribute

Example: *instructor* = (ID, name, dept_name, salary)

- **Def:** $R = (A_1, A_2, \dots, A_n)$ is a **relation schema**
- **Def:** A **relation instance** r is defined over schema R , denoted by $r(R)$
 - The current values $r(R)$ are specified by a **table**
 - An element t of relation r is called a **tuple**, denoted by a **row** in table

Attributes

- **Def:** The **domain** of the attribute is the set of allowed values
- 1. Attribute values are (normally) required to be **atomic, 1NF**
 - **indivisible** , 2020211311456
- 2. The special value **null** is a member of every domain
 - indicating that the value is unknown
 - The null value causes complications in definition of operations, e.g. **index failure**

Relations are Unordered

- Order of tuples is irrelevant
 - tuples may be stored in an arbitrary order
 - Example: relation *instructor* with unordered tuples
- Attribute orders are also irrelevant
 - $(A_1, A_2, \dots, A_n) = (A_2, A_1, \dots, A_n)$



<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000

2.2 Database Schema

- **Def: Database schema**

- is the **logical structure** of database.

- **Def: Database instance**

- is a **snapshot** of data in database at a given instant in time.

- Example:

- schema: *instructor* (ID, name, dept_name, salary)

- Instance:

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000

2.3 Keys

- **Key**: 键, 码
- **Def: Super key** is a set of one or more attributes, that can be used to **identify uniquely** a tuple in the relation
- **Def:** Given relation schema $R(A_1, A_2, \dots, A_n)$, a subset K is **super key**, if no two distinct tuples have the same values on all attributes in K
 - if t_1 and t_2 are in r ($t_1 \neq t_2$), then $t_1.K \neq t_2.K$
 - the super key may contain **redundant** attributes
e.g. $\{ID\}$ vs $\{ID, name, dept_name\}$
- **Def:** K is a **candidate key** if K is **minimal** super key
 - e.g. $\{ID\}$ is a candidate key for *instructor*

Keys (cont.)

- A relation may have **several** candidate keys
 - **Def** : **Primary key** is a candidate key as **principal means** to identify tuples
 - e.g. *ID* in *instructor*
 - **Integrity of primary key**
 - values on the primary key are not permitted to be **null**
 - **Def** : Primary attributes(**主属性**)
 - the attributes in the **candidate keys**, i.e. the element of the primary key, is called the primary attributes
 - non-primary attributes(**非主属性**)
- **Note:** 最好选取数值型 (int, float) 候选键作为主键，便于提高基于主键的查询速度 (不要选字符串型属性，如 varchar、datetime, e.g. studentname)

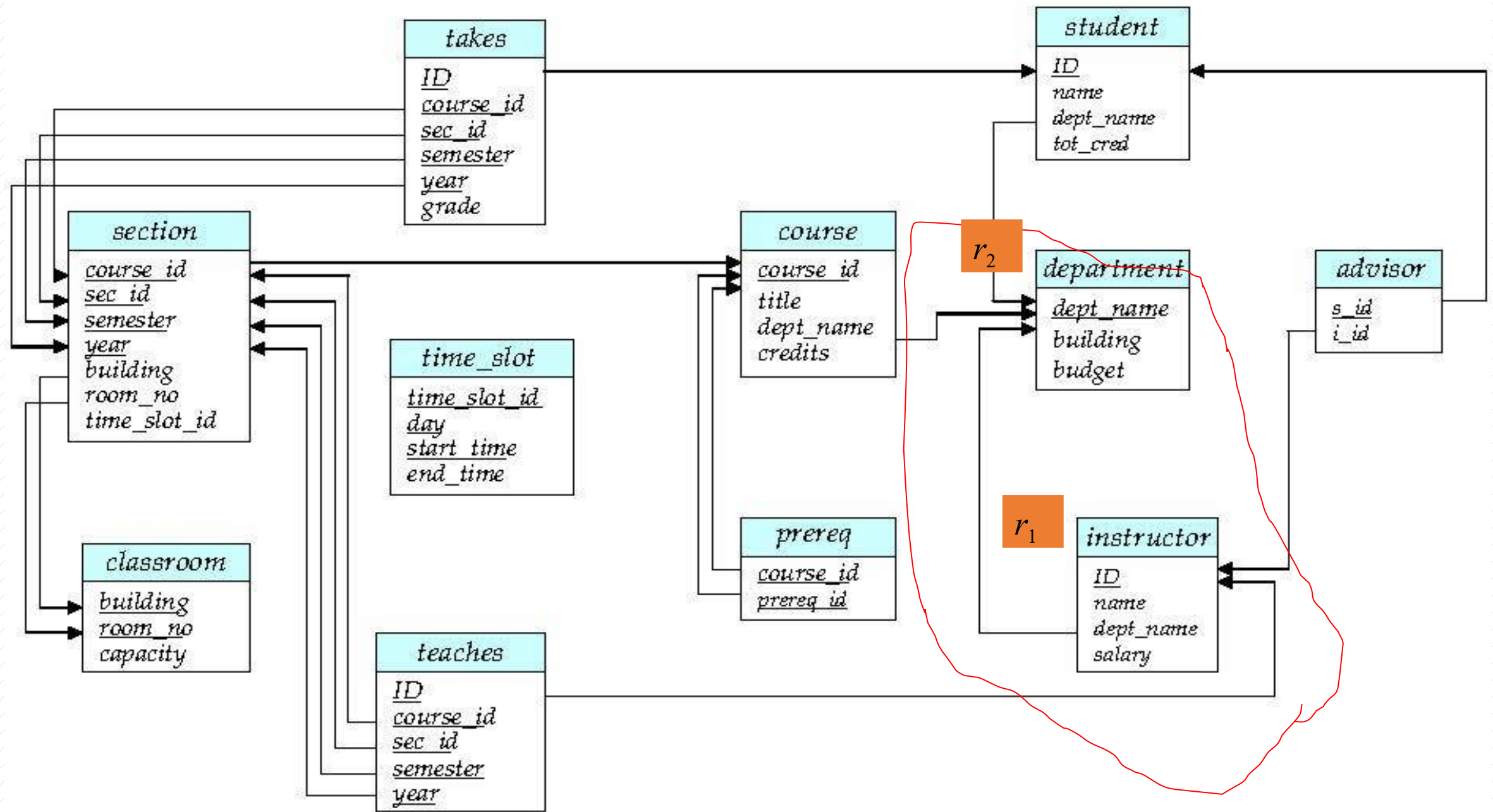


Fig. 2.9 Schema Diagram

Foreign key

- **Describing relationship** among relations, a *instructor* belongs to *department* 
- **Def** : If X is one or more attributes in schema r_1 , e.g. *instructor*, and X is also the **primary-key** of another schema r_2 , e.g. *department*
 - X is called a **foreign key** from r_1 referencing r_2
 - 由 r_1 参照 / 关联 r_2 的外键
 - r_1 is called the **referencing relation** of the foreign key dependency
 - 外键的参照 / 关联关系
 - r_2 is called the **referenced relation** of the foreign key
 - 外键的 被参照 / 关联关系
 - e.g. if $X = dept_name$ in $r_1 = instructor$, $r_2 = department$, then *dept_name* in *instructor* is a **foreign key** from *instructor* referencing *department*

Foreign key/Referential Integrity

- **注意事项：**
 - 数据库表之间存在外键关系，导入数据时，**先导入被参照关系 r_2** (*department*) 中的数据，**再导入参照关系 r_1** (*instructor*) 数据
 - 先定义各个 *department*，再导入属于各个 *department* 的 *instructor* 的数据

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
69987	White	null	null

instructor

dept_name	building	budget
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000
null	Taylor	null

department

Foreign key/Referential Integrity

■ Referential Integrity

- values appearing in **foreign key** attributes (*dept_name*) of any tuples in the **referencing** relation r_1 (*instructor*) **must** appear in attributes of at least **one tuples** in the **referenced** relation r_2 (*department*) 
- why?
 - the department that an instructor belongs to *instructor* (**Comp. Sci**) must be a department that **defined** in *department* corresponding to a tuple (**Comp. Sci**) in *department*

2.4 Schema Diagram for University Database

- A database schema, with primary key and foreign key dependency, can be depicted as schema diagram
 - ▣ **nodes**: schema, attributes, primary key
 - ▣ **arcs**: foreign key dependency, from referencing relation r_1 to referenced relation r_2

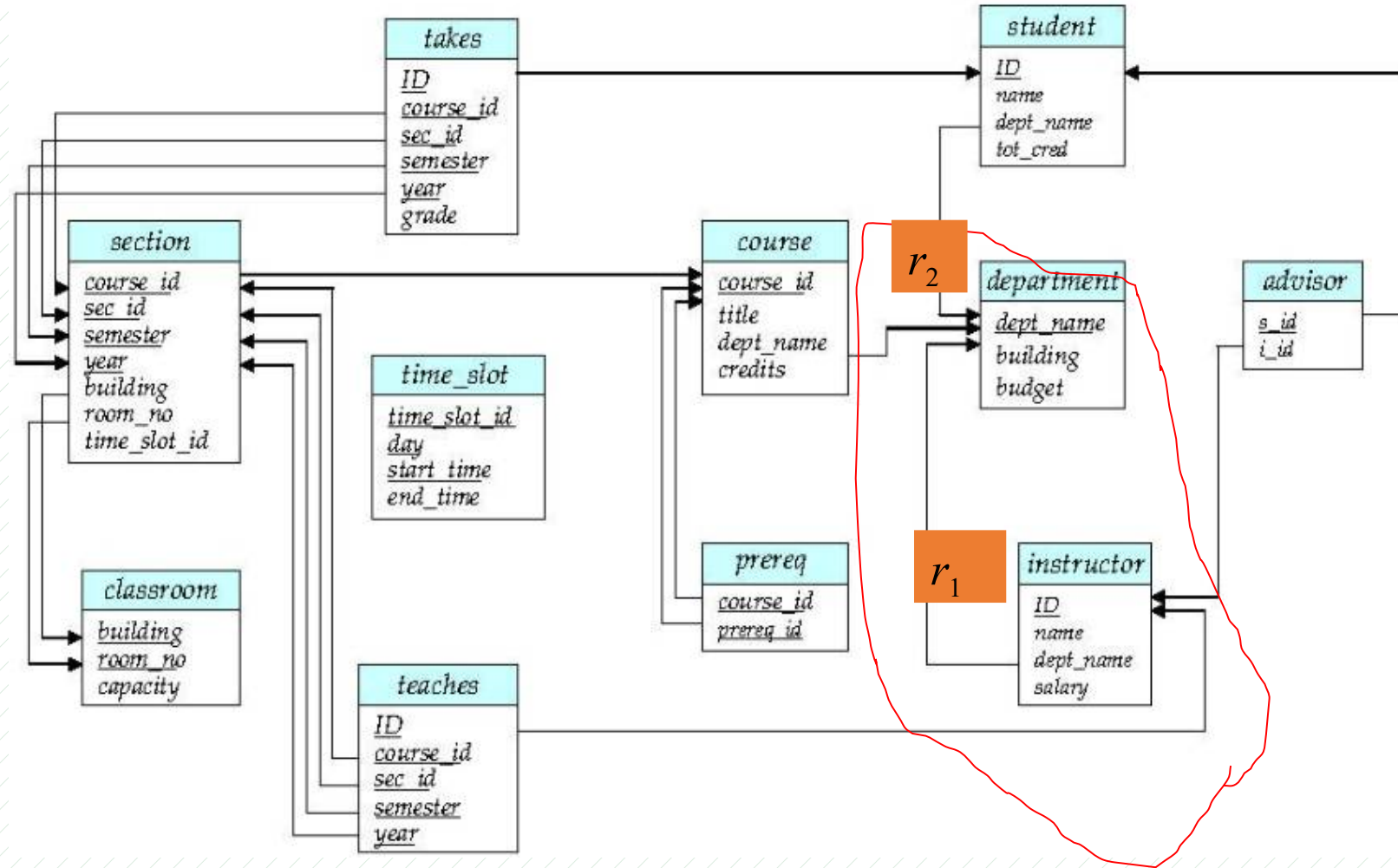


Fig. 2.9 Schema Diagram

2.5 Relational Query Languages

- **Procedural** versus **non-procedural**, or declarative
- **Pure languages:**
 - 1.relational algebra
 - 2.tuple relational calculus
 - 3.domain relational calculus
- The above pure languages are **equivalent**
- We will concentrate on **relational algebra**
 - consists of **6** basic operations

2.6 Relational Algebra

- **Procedural** language consisting of a set of operations/operators that take **one or two relations** as input and produce **a new relation** as output.
- Six basic operators
 - ▣ select: σ
 - ▣ project: Π
 - ▣ Cartesian product: $\times$
 - ▣ union: $\cup$
 - ▣ set difference: $-$
 - ▣ rename: ρ

Join is not a basic operator

Symbol (Name)	Example of Use
σ (Selection)	$\sigma_{\text{salary} \geq 85000}(\text{instructor})$
	Return rows of the input relation that satisfy the predicate.
Π (Projection)	$\Pi_{ID, salary}(\text{instructor})$
	Output specified attributes from all rows of the input relation. Remove duplicate tuples from the output.
$\bowtie$ (Natural join)	$\text{instructor} \bowtie \text{department}$
	Output pairs of rows from the two input relations that have the same value on all attributes that have the same name.
$\times$ (Cartesian product)	$\text{instructor} \times \text{department}$
	Output all pairs of rows from the two input relations (regardless of whether or not they have the same values on common attributes)
$\cup$ (Union)	$\Pi_{name}(\text{instructor}) \cup \Pi_{name}(\text{student})$
	Output the union of tuples from the two input relations.

Relational algebra expression

- **Def:** Relational algebra expression E is defined as follows
 - a **basic expression** is
 - a relation r in DB, or
 - a constant relation $C = \{r_1, r_2, \dots, r_n\}$, written by listing its tuples
 - a **general expression** is defined:
 - if E_1 and E_2 are relational algebra expressions, we have
 - $E_1 \cup E_2$
 - $E_1 - E_2$
 - $E_1 \bowtie E_2$
 - $\sigma_p(E_1)$
 - $\Pi_s(E_1)$,
 - $\rho_x(E_1)$

1. Select Operation

- **Def:** The **select** operation selects **tuples** that satisfy a given predicate.
- Notation: $\sigma_p(r)$
- p is called the **selection predicate**
- Example: select those **tuples** of the *instructor* relation where the instructor is in Physics department.

- Query

$\sigma_{dept_name=Physics}(instructor)$

Select *
From *r*
Where *p*

- Result

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
33456	Gold	Physics	87000

Select Operation (Cont.)

- Allow **comparisons**

$=, \neq, >, \geq, <, \leq$

in the selection predicate.

- **Combine** several predicates into a larger predicate by using connectives:

$\wedge$ (**and**), $\vee$ (**or**), $\neg$ (**not**)

- Example: Find the instructors in Physics with a salary greater \$90,000:

$\sigma_{dept_name="Physics" \wedge salary > 90,000} (instructor)$

- The select predicate may include comparisons between two attributes.

- Example, find all departments whose name is the same as their building name:

$\sigma_{dept_name=building} (department)$

2. Project Operation

- **Def:** Operation that returns its argument relation, with certain attributes **left out**.
- Notation:

$$\Pi_{A_1, A_2, A_3 \dots A_k}(r)$$

where $A_1, A_2, \dots, A_k$ are attribute names and r is a relation name.

- The result defined as k columns by **erasing** the columns that are not listed.
- **Duplicate** rows are not removed from the result, since relations are sets

```
Select A1, A2, ..., Ak
From r
```

Project Operation Example

- Example: eliminate the *dept_name* attribute of *instructor*
- Query:

$\Pi_{ID, name, salary}(instructor)$

- Result:

<i>ID</i>	<i>name</i>	<i>salary</i>
10101	Srinivasan	65000
12121	Wu	90000
15151	Mozart	40000
22222	Einstein	95000
32343	El Said	60000
33456	Gold	87000
45565	Katz	75000
58583	Califieri	62000

Composition of Relational Operations

- The result of a relational-algebra operation is relation
- Relational-algebra operations can be **composed** together into a **relational-algebra expression**.
- Example-- Find the names of all instructors in Physics department.

$$\Pi_{name}(\sigma_{dept_name = "Physics"}(instructor))$$

- Instead of giving the name of a relation as the argument of projection **operation**, we give an **expression** that evaluates to a relation.

3. Cartesian-Product Operation

- **Def:** Cartesian-product operation (denoted by $\times$) allows to **combine** information from two relations.
- Example: Cartesian product of *instructor* and *teaches* is written as:
instructor $\times$ *teaches*
- Construct a tuple of the result out of each possible pair of tuples: one from *instructor*, and another from *teaches*
- **Note:** Since instructor *ID* appears in both relations, we distinguish between these attributes by attaching to the attribute the name of relation from which attribute originally came.
 - *instructor.ID*
 - *teaches.ID*

```
Select *  
From instructor, teaches
```

instructor X teaches

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

<i>ID</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>
10101	CS-101	1	Fall	2017
10101	CS-315	1	Spring	2018
10101	CS-347	1	Fall	2017
12121	FIN-201	1	Spring	2018
15151	MU-199	1	Spring	2018
22222	PHY-101	1	Fall	2017
32343	HIS-351	1	Spring	2018
45565	CS-101	1	Spring	2018
45565	CS-319	1	Spring	2018
76766	BIO-101	1	Summer	2017
76766	BIO-301	1	Summer	2018
83821	CS-190	1	Spring	2017

The table: *instructor x teaches*

<i>instructor.ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>	<i>teaches.ID</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	12121	FIN-201	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	15151	MU-199	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
12121	Wu	Finance	90000	10101	CS-101	1	Fall	2017
12121	Wu	Finance	90000	10101	CS-315	1	Spring	2018
12121	Wu	Finance	90000	10101	CS-347	1	Fall	2017
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2018
12121	Wu	Finance	90000	15151	MU-199	1	Spring	2018
12121	Wu	Finance	90000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...

4. Join Operation

- The Cartesian-Product

instructor X teaches

associates every tuple of instructor with every tuple of teaches.

- Most of the resulting rows have information about instructors who did **NOT** teach a particular course.
- To get only those tuples of ***instructor X teaches*** that pertain to instructors and the courses that they taught, we write:

$\sigma_{\text{instructor.id} = \text{teaches.id}} (\text{instructor} \times \text{teaches})$

- only those tuples of “*instructor X teaches*” that pertain to instructors and the courses that they taught.

Join Operation (Cont.)

- The table corresponding to:

$\sigma_{instructor.id = teaches.id}(instructor \times teaches)$

Select *

From *instructor* natural join *teaches*

<i>instructor.ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>	<i>teaches.ID</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2017
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2018
15151	Mozart	Music	40000	15151	MU-199	1	Spring	2018
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2017
32343	El Said	History	60000	32343	HIS-351	1	Spring	2018
45565	Katz	Comp. Sci.	75000	45565	CS-101	1	Spring	2018
45565	Katz	Comp. Sci.	75000	45565	CS-319	1	Spring	2018
76766	Crick	Biology	72000	76766	BIO-101	1	Summer	2017
76766	Crick	Biology	72000	76766	BIO-301	1	Summer	2018
83821	Brandt	Comp. Sci.	92000	83821	CS-190	1	Spring	2017
83821	Brandt	Comp. Sci.	92000	83821	CS-190	2	Spring	2017
83821	Brandt	Comp. Sci.	92000	83821	CS-319	2	Spring	2018

Join Operation (Cont.)

- **Def:** The **join operation** **combine select operation** and **Cartesian-Product** operation into a single operation.
- **Def:** Consider relations $r (R)$ and $s (S)$
- Let θ be a **predicate** on attributes in schema R union S .
The join operation $r \bowtie_{\theta} s$ is defined as follows:

$$r \bowtie_{\theta} s = \sigma_{\theta} (r \times s)$$

➤ Thus

$$\sigma_{instructor.id = teaches.id} (instructor \times teaches)$$

Can equivalently be written as

$$instructor \bowtie_{Instructor.id = teaches.id} teaches.$$

5. Union Operation

- The union operation allows us to combine two relations
 - Notation: $r \cup s$
- For $r \cup s$ to be valid.
 1. r, s must have the **same arity** (same number of attributes)
 2. The attribute **domains** must be **compatible**
(the 2nd column of r deals with the same type of values as the 2nd column of s)
- Example: Find all courses taught in the Fall 2017 semester, **or** in the Spring 2018 semester, **or** in both

$$\Pi_{\text{course_id}} (\sigma_{\text{semester}=\text{"Fall"} \wedge \text{year}=2017}(\text{section})) \cup \Pi_{\text{course_id}} (\sigma_{\text{semester}=\text{"Spring"} \wedge \text{year}=2018}(\text{section}))$$

Union Operation (Cont.)

- Result of:

$$\Pi_{course_id} (\sigma_{semester="Fall" \wedge year=2017} (section)) \cup$$
$$\Pi_{course_id} (\sigma_{semester="Spring" \wedge year=2018} (section))$$

<i>course_id</i>
CS-101
CS-315
CS-319
CS-347
FIN-201
HIS-351
MU-199
PHY-101

6. Set-Intersection Operation

- The set-intersection operation allows us to find tuples that are in both input relations.
- Notation: $r \cap s$
- Assume:
 - r, s have the *same arity*
 - attributes of r and s are *compatible*
- Example: Find all courses taught in both Fall 2017 and Spring 2018 semesters.

$$\Pi_{course_id} (\sigma_{semester="Fall" \wedge year=2017}(section)) \cap \Pi_{course_id} (\sigma_{semester="Spring" \wedge year=2018}(section))$$

- Result

<i>course_id</i>
CS-101

7. Set Difference Operation

- The set-difference operation allows find tuples that are in one relation but are not in another.
- Notation $r - s$
- Set differences must be taken between **compatible** relations.
 - r and s must have the **same** arity
 - attribute domains of r and s must be compatible
- Example: find all courses taught in Fall 2017, but not in Spring 2018

$$\Pi_{course_id} (\sigma_{semester="Fall" \wedge year=2017} (section)) - \Pi_{course_id} (\sigma_{semester="Spring" \wedge year=2018} (section))$$

<i>course_id</i>
CS-347
PHY-101

8. The Assignment Operation

- The **assignment operation** is denoted by $\leftarrow$
- **Working like** assignment in programming language.
- Example: Find all instructor in Physics and Music department.

$Physics \leftarrow \sigma_{dept_name="Physics"}(instructor)$

$Music \leftarrow \sigma_{dept_name="Music"}(instructor)$

$Physics \cup Music$

9. The Rename Operation

- The results of relational-algebra expressions do not have a name. The rename operator ρ is provided for a name
- The expression:

$$\rho_x(E)$$

returns the result of expression E under the name x

- Another form of the rename operation:

$$\rho_{x(A1,A2, \dots An)}(E)$$

Equivalent Queries

Find courses taught by instructors in Physics with salary greater than 90,000

- Query 1

$$\sigma_{dept_name="Physics" \wedge salary > 90,000} (instructor)$$

- Query 2

$$\sigma_{dept_name="Physics"} (\sigma_{salary > 90,000} (instructor))$$

- The two queries are not identical, however, they are equivalent
 - they give the same result on any database

Equivalent Queries

- Example: Find courses taught by instructors in Physics

- Query 1

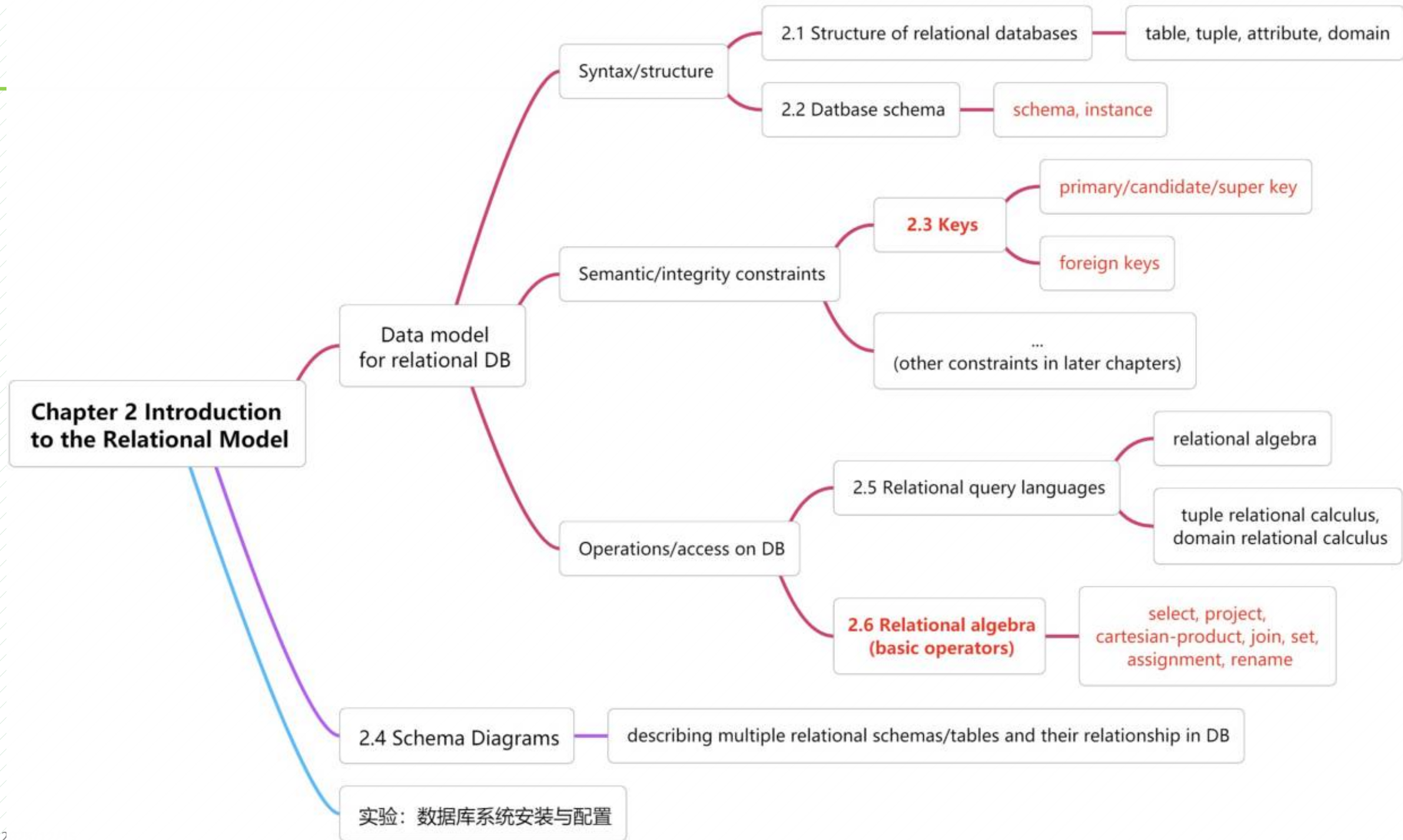
$\sigma_{dept_name = \text{"Physics"}} (instructor \bowtie_{instructor.ID = teaches.ID} teaches)$

- Query 2

$(\sigma_{dept_name = \text{"Physics"}} (instructor)) \bowtie_{instructor.ID = teaches.ID} teaches$

- The two queries are equivalent
-- they give the same result on any database.

查询结果相同，执行效率 / 成本不同◇查询优化





Thanks for your
attention